RAG 101

Chunking Strategies

Why, When, and How to chunk for enhanced RAG. Build intuition to develop chunking strategies.

Shanmukha Ranganath · October 5, 2024 · 12 min read

UNLOCK THE FULL POTENTIAL OF YOUR RAG WORKFLOW

Why, When, and How to chunk for enhanced RAG

> *Figure: How do we split the balls? (Generated using Canva)*

The maximum number of tokens that a Large Language Model can process in a single request is known as **context length** (or context window). The table below shows the context length for all versions of GPT-4 (as of Sep 2024). While context lengths have been increasing with every iteration and with every newer model, there remains a limit to the information we can provide the model. Moreover, there is an inverse correlation between the size of the input and the context relevancy of the responses generated by the LLM, short and focused inputs produce better results than long contexts containing vast information. This emphasizes the importance of breaking down our data into smaller, relevant chunks to ensure more appropriate responses from the LLMs—at least until LLMs can handle enormous amounts of data without re-training.

> *Figure: Table of GPT-4 context lengths (image in original article)*

The Context Window represented in the image is inclusive of both input and output tokens.

> Learn this step by step with the interactive **Machine Learning roadmap**.

Why Does Context Length Matter

Though longer contexts give a more holistic picture to the model and help it in understanding relationships and make better inferences, shorter contexts on the other hand reduce the amount of data that the model needs to understand and thus decreases latency, making the model more responsive. It also helps in minimizing hallucinations of the LLM since only the relevant data is given to the model. So, it's a balance between performance, efficiency, and how complex our data is and, we need to run experiments on how much data is the right amount of data that yields best results with reasonable resources.

GPT-4 model's 128k tokens may seem like a lot, so let's convert them to actual words and put them in perspective. From the OpenAI Tokenizer:

> *A helpful rule of thumb is that one token generally corresponds to ~4 characters of text for common English text. This translates to roughly ¾ of a word (so 100 tokens ~= 75 words).*

Let's consider *The Hound of the Baskervilles* by Arthur Conan Doyle (Project Gutenberg License) as our example throughout this article. This book is 7734 lines long with 62303 words, which comes to approximately 83,700 tokens.

If you are interested in exactly calculating tokens and not just approximation, you can use OpenAI's `tiktoken`:

PYTHON

```
import request.
from tiktoken import encoding_for_model
url = "https://www.gutenberg.org/cache/epub/3070/pg3070.txt"
response = requests.get(url)
if response.status_code == 200:
    book_full_text = response.text
encoder = encoding_for_model("gpt-4o")
tokens = encoder.encode(book_full_text)
print(f"Number of tokens: {len(tokens)}")
```

Which gives the number of tokens to be `Number of tokens: 82069`

What is Chunking

Figure: Chunking Cheese!! (Generated using Canva)

I like the wiki definition of chunking, as it applies to RAG as much as it is true in cognitive psychology.

> *Chunking is a process by which small individual pieces of a set of information are bound together. The chunks are meant to improve short-term retention of the material, thus bypassing the limited capacity of working memory and allowing the working memory to be more efficient.*

The process of splitting large datasets into smaller, meaningful pieces of information so that the LLM's non-parametric memory can be used more effectively is called chunking. There are many different ways to split the data to improve retrieval of chunks for RAG, and we need to choose depending on the type of data that is being consumed.

Chunking is a crucial pre-retrieval step in the RAG pipeline that directly influences the retrieval process and significantly affects the final output. In this article, we will look at the most common strategies of chunking and evaluate them for retrieval metrics in the context of our data.

Intuition for a splitter

Instead of going over existing chunking strategies/splitters available in different libraries right away, let's start building a simple splitter and explore the important aspects that needs to be considered, to build the intuition of writing a new splitter. Let's start with a basic splitter and progressively improve it by solving its drawbacks/limitations.

1. Naive Chunking

When we talk about splitting data, the first thing that comes to our mind is to split it at newline character. Lets go ahead with the implementation. But as you can see it leaves lots of return carriage characters. Also, we just assumed `n` and `r` since we are only dealing with the English language, but what if we want to parse other languages? Let's add the flexibility to pass in the characters to split as well.

PYTHON

```
def naive_splitter_v2(text: str, separators: List[str] = ["n", "r"]) -> List[str]:
    """Splits text at every separator"""
    splits = [text]
    for sep in separators:
        splits = [segment for part in result for segment in part.split(sep) if segment]
    return splits
```

Figure: output of naive_splitter_v2

You might've already guessed from the output why we call this method **Naive**. The idea has lots of drawbacks:

- **No Chunk limits.** As long as a line has one of the delimiter, it will break, but what if we a chunk doesn't have those delimiters, it could go to any length.
- Similarly, as you can clearly see in the output, there are chunks that are **too small!** a single word chunks doesn't make any sense without surrounding context.
- **Breaks in between lines:** A chunk is retrieved based on the question that is asked, but a sentence/line is totally incomplete or even has different meaning if we truncate it mid sentence.

Let's try to fix these problems one by one.

2. Fixed Window Chunking

Let's first tackle the first problem of too long or too short chunk sizes. This time we take in a limit for the size and try to split the text exactly when we reach the size.

PYTHON

```
def fixed_window_splitter(text: str, chunk_size: int = 1000) -> List[str]:
    """Splits text at given chunk_size"""
    splits = []
    for i in range(0, len(text), chunk_size):
        splits.append(text[i:i + chunk_size])
    return splits
```

Figure: output of fixed_window_splitter

We did solve the minimum and maximum bounds of the chunk, since it is always going to be `chunk_size`. But the breaks in between words still remains the same. From the output we can see, we are losing the meaning of a chunk since it is split mid-sentence.

3. Fixed Window with Overlap Chunking

The easiest way to make sure that we don't split in between words is to make sure we go over until the end of the word and then stop. Though this will make the context not too long and within the the expected chunksize range, a better approach would be to start the next chunk some *x* characters/words/tokens behind the actual start position, so that the context is always preserved and will be continuous.

PYTHON

```
def fixed_window_with_overlap_splitter(text: str, chunk_size: int = 1000, chunk_overlap: int = 10)
    -> List[str]:
    """Splits text at given chunk_size, and starts next chunk from start - chunk_overlap
        position"""
    chunks = []
    start = 0
    while start <= len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start = end - chunk_overlap
    return chunks
```

Figure: output of fixed_window_with_overlap_splitter

4. Recursive Character Chunking

With Chunk Size and Chunk Overlap fixed, we can now solve the problem of mid-word or mid-sentence splitting. This can be solved with a bit of modification to our initial Naive splitter. We take a list of separators and pick a good separator as we grow more to the chunk size. Meanwhile we will still continue to use the chunk overlap the same way. This is one of the most popular splitters available in LangChain package called `RecursiveCharacterTextSplitter`. This works the same way we approached:

- Starts with highest priority separator, which starts from beginning `nn` and moves to next in the separators list.
- If a split exceeds the `chunk_size`, it applies the next separator until the current split falls under the correct size.
- The next split starts `chunk_overlap` characters behind the current split ending, thus maintaining the continuity of the context.

Figure: output of recursive_character_splitter

4. Semantic Chunking

So far, we've only considered *where* to split our data, whether it's at end of a paragraph or a new line or a tab or other separators. But we haven't thought about *when* to split, that is, how to better capture a meaningful chunk rather than just a chunk of some length. This approach is known as semantic chunking. Let's use **Flair** to detect sentence boundaries or specific entities and create meaningful chunks. The text is split into sentences using `SegtokSentenceSplitter`, which ensures it is divided at meaningful boundaries. We keep the sizing logic the same, to group until we reach `chunk_size` and overlap of `chunk_overlap` to ensure context is maintained.

PYTHON

```python
def semantic_splitter(text: str, chunk_size: int = 1000, chunk_overlap: int = 10) -> List[str]:
    from flair.models import SequenceTagger
    from flair.data import Sentence
    from flair.splitter import SegtokSentenceSplitter
    splitter = SegtokSentenceSplitter()
    # Split text into sentences
    sentences = splitter.split(text)
    chunks = []
    current_chunk = ""
    for sentence in sentences:
        # Add sentence to the current chunk
        if len(current_chunk) + len(sentence.to_plain_string()) <= chunk_size:
            current_chunk += " " + sentence.to_plain_string()
        else:
            # If adding the next sentence exceeds max size, start a new chunk
            chunks.append(current_chunk.strip())
            current_chunk = sentence.to_plain_string()
    # Add the last chunk if it exists
    if current_chunk:
        chunks.append(current_chunk.strip())
    return chunks
```

Figure: output of semantic_splitter

LangChain has two such splitters, using the NLTK and spaCy libraries, so do check them out.

So, generally, in static chunking methods, **Chunk Size** and **Chunk Overlap** are two major factors to consider while determining chunking strategy. Chunk size is the number of character/words/tokens of each chunk and chunk overlap is the amount of previous chunk to be included in the current chunk so the context is continuous. Chunk overlap can also be a expressed as number of character/words/tokens or a percentage of chunk size.

You can use the cool **ChunkViz** tool to visualize how different chunking strategies behave with different chunk size and overlap parameters:

Figure: Hound Of Baskervilles on ChunkViz

5. Embedding Chunking

Even though Semantic chunking gets the job done, NLTK, spaCy, or Flair use their own models/embeddings to understand the given data and try to give us when best the data can be split semantically. When we move on to our actual RAG implementation, our embeddings might be different from the ones that our chunks are merged together with and hence can be understood in a different way altogether. So, in this approach we start off splitting to sentences and form the chunks based on the same embedding model we later are going to use for our RAG retrieval. To do things differently, we will use NLTK for in this to split to sentences and us OpenAIEmbeddings to merge them to form sentences.

PYTHON

```python
def embedding_splitter(text_data, chunk_size=400):
    import os
    import nltk
    from langchain_openai.embeddings import AzureOpenAIEmbeddings
    from sklearn.metrics.pairwise import cosine_similarity
    import numpy as np
    from dotenv import load_dotenv, find_dotenv
    from tqdm import tqdm
```

```
from flair.splitter import SegtokSentenceSplitter
load_dotenv(find_dotenv())
# Set Azure OpenAI API environment variables (ensure these are set in your environment)
# You can also set these in your environment directly
# os.environ["OPENAI_API_KEY"] = "your-azure-openai-api-key"
# os.environ["OPENAI_API_BASE"] = "your-azure-openai-api-endpoint"
os.environ["OPENAI_API_VERSION"] = "2023-05-15"
# Initialize OpenAIEmbeddings using LangChain's Azure support
embedding_model = AzureOpenAIEmbeddings(deployment="text-embedding-ada-002-01")  # Use your
    Azure model name
# Step 1: Split the text into sentences
def split_into_sentences(text):
    splitter = SegtokSentenceSplitter()
    # Split text into sentences
    sentences = splitter.split(text)
    sentence_str = []
    for sentence in sentences:
        sentence_str.append(sentence.to_plain_string())
    return sentence_str[:100]
# Step 2: Get embeddings for each sentence using the same Azure embedding model
def get_embeddings(sentences):
    embeddings = []
    for sentence in tqdm(sentences, desc="Generating embeddings"):
        embedding = embedding_model.embed_documents([sentence])  # Embeds a single sentence
        embeddings.append(embedding[0])  # embed_documents returns a list, so take the first
            element
    return embeddings
# Step 3: Form chunks based on sentence embeddings, a similarity threshold, and a max chunk
    character size
def form_chunks(sentences, embeddings, similarity_threshold=0.7, chunk_size=500):
    chunks = []
    current_chunk = []
    current_chunk_emb = []
    current_chunk_length = 0  # Track the character length of the current chunk
    for i, (sentence, emb) in enumerate(zip(sentences, embeddings)):
        emb = np.array(emb)  # Ensure the embedding is a numpy array
        sentence_length = len(sentence)  # Calculate the length of the sentence
        if current_chunk:
            # Calculate similarity with the current chunk's embedding (mean of embeddings in
                the chunk)
            chunk_emb = np.mean(np.array(current_chunk_emb), axis=0).reshape(1, -1)  # Average
                embedding of the chunk
            similarity = cosine_similarity(emb.reshape(1, -1), chunk_emb)[0][0]
            if similarity < similarity_threshold or current_chunk_length + sentence_length >
                chunk_size:
                # If similarity is below threshold or adding this sentence exceeds max chunk
                    size, create a new chunk
                chunks.append(current_chunk)
                current_chunk = [sentence]
                current_chunk_emb = [emb]
                current_chunk_length = sentence_length  # Reset chunk length
            else:
                # Else, add sentence to the current chunk
                current_chunk.append(sentence)
                current_chunk_emb.append(emb)
                current_chunk_length += sentence_length  # Update chunk length
        else:
            current_chunk.append(sentence)
            current_chunk_emb = [emb]
            current_chunk_length = sentence_length  # Set initial chunk length
    # Add the last chunk
    if current_chunk:
        chunks.append(current_chunk)
    return chunks
# Apply the sentence splitting
sentences = split_into_sentences(text_data)
# Get sentence embeddings
embeddings = get_embeddings(sentences)
# Form chunks based on embeddings
chunks = form_chunks(sentences, embeddings, chunk_size=chunk_size)
```

```
    return chunks
```

Figure: output of embedding_splitter

6. Agentic Chunking

Our Embedding Chunking should come closer to splitting the data with the cosine similarity of the embeddings created. Though this works well, we have one major drawback: it doesn't understand the semantics of the text. "I Like You" vs "I Like You" with sarcasm on "like," both sentences will have the same embeddings and hence will correspond to the same cosine distance when calculated. This is where Agentic (or LLM-based) chunking comes in handy. It analyzes the content to identify points to break logically based on standalone-ness and semantic coherence.

PYTHON

```
def agentic_chunking(text_data):
    from langchain_openai import AzureChatOpenAI
    from langchain.prompts import PromptTemplate
    from langchain
    llm = AzureChatOpenAI(model="gpt-4o",
                          api_version="2023-03-15-preview",
                          verbose=True,
                          temperature=1)
    prompt = """I am providing a document below.
    Please split the document into chunks that maintain semantic coherence and ensure that each
        chunk represents a complete and meaningful unit of information.
    Each chunk should stand alone, preserving the context and meaning without splitting key ideas
        across chunks.
    Use your understanding of the content's structure, topics, and flow to identify natural
        breakpoints in the text.
    Ensure that no chunk exceeds 1000 characters length, and prioritize keeping related concepts
        or sections together.
    Do not modify the document, just split to chunks and return them as an array of strings, where
        each string is one chunk of the document.
    Return the entire book not dont stop in betweek some sentences.
    Document:
    {document}
    """
    prompt_template = PromptTemplate.from_template(prompt)
    chain = prompt_template | llm
    result = chain.invoke({"document": text_data})
    return result
```

Evaluation

We will cover the RAG evaluation techniques in an upcoming post; in this post we will see two metrics defined by RAGAS, `context_precision` and `context_relevance`, that determine how our chunking strategies performed.

> **Context Precision** is a metric that evaluates whether all of the ground-truth relevant items present in the contexts are ranked higher or not. Ideally all the relevant chunks must appear at the top ranks. This metric is computed using the `question`, `ground_truth` and the `contexts`, with values ranging between 0 and 1, where higher scores indicate better precision.

> **Context Relevancy** gauges the relevancy of the retrieved context, calculated based on both the question and contexts. The values fall within the range of (0, 1), with higher values indicating better relevancy.

In the next article we will go over proposal retrieval, one of the agentic splitting methods, and calculate RAGAS metrics for all out strategies.

Conclusion

In this article we've covered why we need chunking and have developed an intuition to build some of the strategies and their implementation as well as their corresponding code in some of the well-known libraries. These are just basic chunking strategies, though newer and newer strategies are being invented every day to make better retrieval even better.